Practical Assessment-3-Technical Documentation:-

Three-Tier Application Deployment with Monitoring on AWS

DevOps Internship Assignment

Company: Modern India Pvt Limited
Role: DevOps Engineer Intern
Project: Full Stack Three-Tier Web Application Deployment
Cloud Platform: Amazon Web Services (AWS)

---

1. Introduction

1.1 Purpose

The purpose of this project is to design, deploy, secure, and monitor a three-tier web application on AWS infrastructure using DevOps best practices. The deployment includes application hosting, database configuration, monitoring, logging, and alerting.

The project demonstrates skills in:

- Cloud infrastructure provisioning
- Application deployment
- Monitoring and observability
- Log aggregation
- Security implementation

1.2 Scope

The scope of the project includes:

- Deploying a **three-tier web application**
- Configuring infrastructure on AWS
- Implementing monitoring using Prometheus and Grafana
- Collecting logs using Loki
- Sending alerts using AWS SNS
- Securing the application with HTTPS

The project does not include advanced production scaling such as load balancing or container orchestration.

1.3 Technology Stack

Layer	Technology
Frontend	React.js
Backend	Node.js
Database	MySQL
Monitoring	Prometheus
Visualization	Grafana
Log Management	Loki + Promtail
Alerting	Alertmanager + AWS SNS
Web Server	Nginx
Cloud Platform	AWS EC2

2. System Architecture

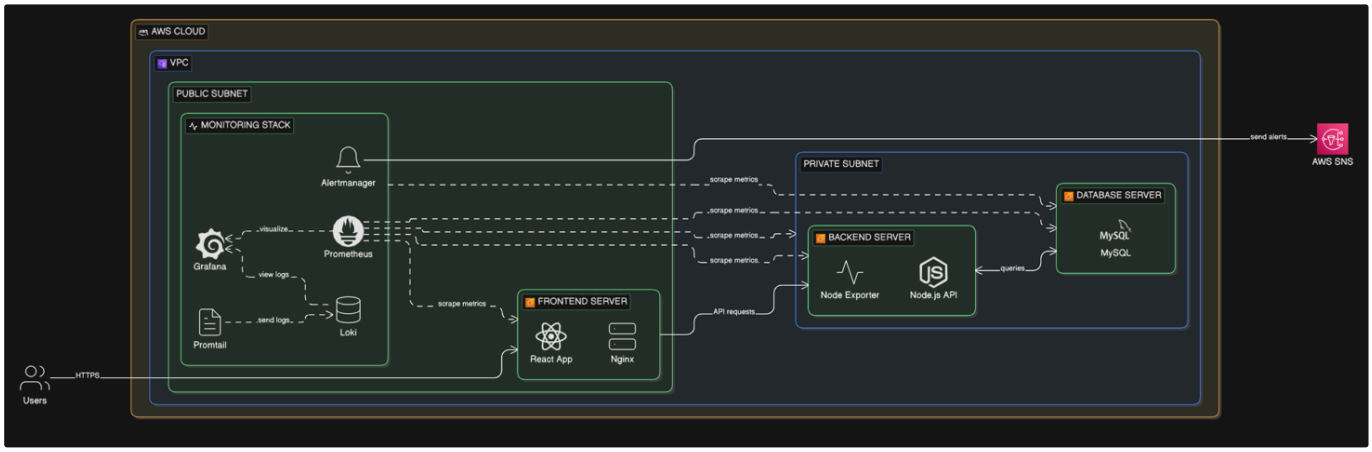
The application follows a **three-tier architecture**.

Architecture Layers

1. **Presentation Layer**
   - React frontend
   - Hosted on Nginx
2. **Application Layer**
   - Node.js backend API
   - Processes business logic
3. **Data Layer**
   - MySQL database server
   - Stores application data

Architecture Flow

```
User
 ↓
Frontend Server (React + Nginx)
 ↓
Backend Server (Node.js API)
 ↓
Database Server (MySQL)
```



---

3. AWS Infrastructure Setup

Three EC2 instances were created.

Server	Type	Purpose
Frontend	Public EC2	React + Nginx
Backend	Private EC2	Node.js API
Database	Private EC2	MySQL

VPC Network Configuration

The infrastructure uses:

- Public Subnet → Frontend server

- Private Subnet → Backend server
- Private Subnet → Database server

Security groups were configured to control traffic between layers.

Security Group Rules

Frontend Server

Port	Purpose
22	SSH
80	HTTP
443	HTTPS
3000	Grafana
9090	Prometheus
3100	Loki

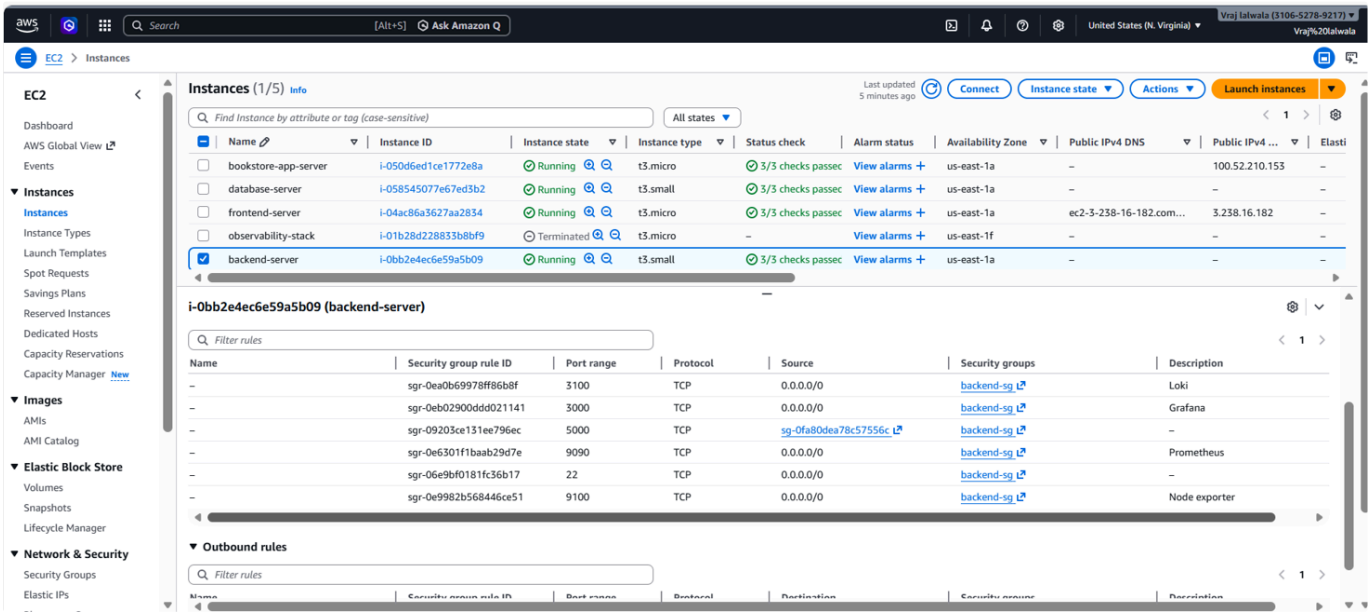
Backend Server

Port	Purpose
22	SSH
5000	Backend API
9100	Node Exporter

Database Server

Port	Purpose
22	SSH
3306	MySQL

Screenshot Placeholder



4. Application Deployment

4.1 Clone Repository

The application source code was cloned from GitHub.

```
git clone https://github.com/pushkar-iamops/3tier-monitoring.git
```

4.2 Frontend Deployment

Steps performed:

1. Installed Node.js
2. Installed dependencies
3. Built the React application
4. Configured Nginx to serve static files

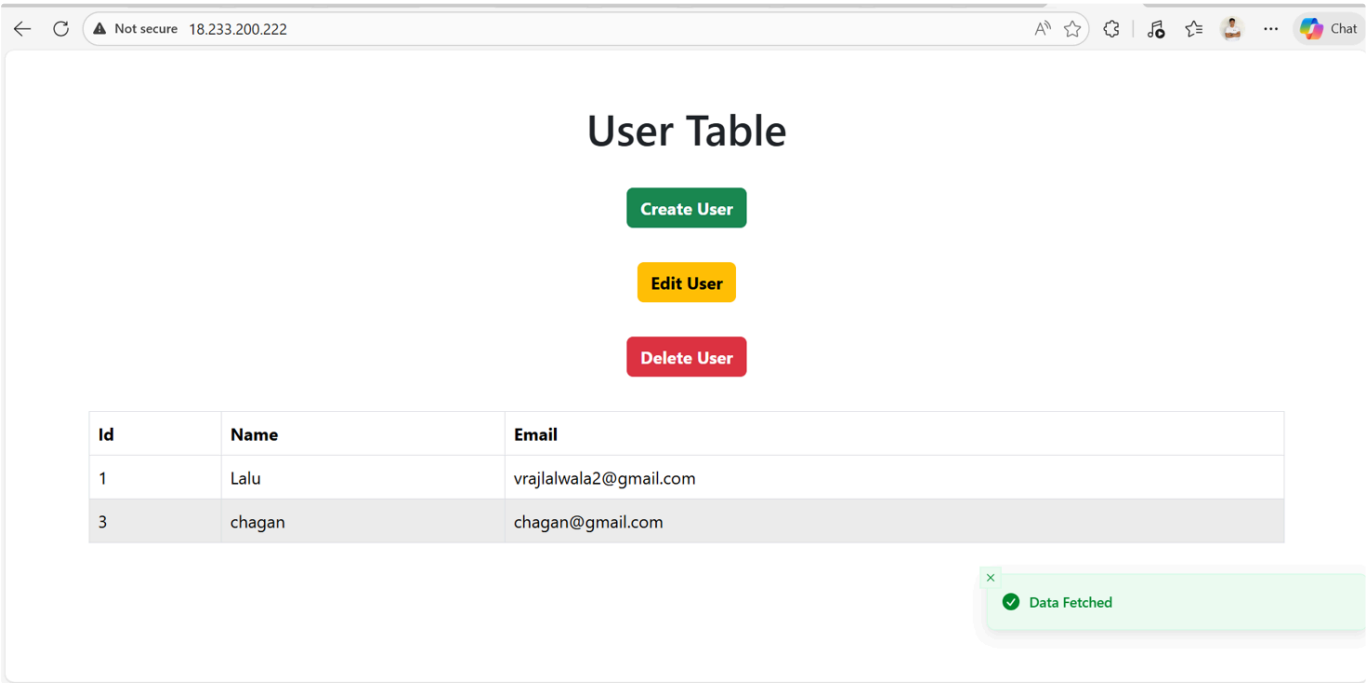
Commands used:

```
cd frontend
npm install
npm run build
sudo cp -r dist/* /var/www/html/
```

```
ubuntu@ip-10-0-1-239:~$ cd 3tier-monitoring/
ubuntu@ip-10-0-1-239:~/3tier-monitoring$ cd frontend/
ubuntu@ip-10-0-1-239:~/3tier-monitoring/frontend$ npm install

added 237 packages, and audited 238 packages in 7s

101 packages are looking for funding
  run `npm fund` for details

11 vulnerabilities (1 low, 5 moderate, 4 high, 1 critical)

To address all issues, run:
  npm audit fix

Run `npm audit` for details.
ubuntu@ip-10-0-1-239:~/3tier-monitoring/frontend$ npm run build

> frontend@0.0.0 build
> vite build

vite v5.2.11 building for production...
✓ 80 modules transformed.
dist/index.html                   1.19 kB │ gzip:  0.63 kB
dist/assets/favicon-Dbkia2L2.ico 68.15 kB
dist/assets/index-CzLSzesQ.css    0.09 kB │ gzip:  0.10 kB
dist/assets/index-BD3uW3oN.js   228.91 kB │ gzip: 74.97 kB
✓ built in 1.76s
ubuntu@ip-10-0-1-239:~/3tier-monitoring/frontend$
```

```
ubuntu@ip-10-0-1-239:~/3tier-monitoring/frontend$ sudo apt install nginx -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  nginx-common
Suggested packages:
  fcgiwrap nginx-doc ssl-cert
The following NEW packages will be installed:
  nginx nginx-common
0 upgraded, 2 newly installed, 0 to remove and 0 not upgraded.
Need to get 565 kB of archives.
After this operation, 1596 kB of additional disk space will be used.
Get:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 nginx-common all 1.24.0-2ubuntu7.6 [43.5 kB]
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 nginx amd64 1.24.0-2ubuntu7.6 [521 kB]
Fetched 565 kB in 0s (21.4 MB/s)
Preconfiguring packages ...
Selecting previously unselected package nginx-common.
(Reading database ... 109535 files and directories currently installed.)
Preparing to unpack .../nginx-common_1.24.0-2ubuntu7.6_all.deb ...
Unpacking nginx-common (1.24.0-2ubuntu7.6) ...
Selecting previously unselected package nginx.
Preparing to unpack .../nginx_1.24.0-2ubuntu7.6_amd64.deb ...
Unpacking nginx (1.24.0-2ubuntu7.6) ...
Setting up nginx-common (1.24.0-2ubuntu7.6) ...
Created symlink /etc/systemd/system/multi-user.target.wants/nginx.service → /usr/lib/systemd/system/nginx.service.
Setting up nginx (1.24.0-2ubuntu7.6) ...
 * Upgrading binary nginx                                                                                    [ OK ]
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for ufw (0.36.2-6) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-10-0-1-239:~/3tier-monitoring/frontend$ sudo systemctl start nginx
ubuntu@ip-10-0-1-239:~/3tier-monitoring/frontend$ sudo systemctl enable nginx
Synchronizing state of nginx.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable nginx
ubuntu@ip-10-0-1-239:~/3tier-monitoring/frontend$
```

Screenshot Placeholder



---

5. Backend Deployment

The backend API server was deployed on a private EC2 instance.

Steps:

1. Installed Node.js
2. Installed dependencies
3. Configured environment variables
4. Started the backend service

```
cd server
npm install
node index.js
```

The backend communicates with the MySQL database server.

```
try {
  //post req to server
  const res = await axios.post("/api", {
    name,
    email
  });

  //incase of success
```

```
    }
    try {
      //put req to server
      const res = await axios.put(`/api`, {
        id,
        name,
        email
      });

      //incase of success
```

```
  const fetchUsers = async () => {
    try {
      const response = await axios.get("/api");
      //setting the empty array as a json object of users got from the server
      setUsers(response.data);

      //using the useRef current which is the same as initialized untill changes
      if (!hasFetchedUsers.current) {
        toast.success("Data Fetched");
        hasFetchedUsers.current = true; // set the ref to true after first fetch
      }
    } catch (error) {
      console.log("There was an error fetching the users!", error);
```

```
        server_name _;

        location / {
                # First attempt to serve request as file, then
                # as directory, then fall back to displaying a 404.
                try_files $uri $uri/ /index.html;
        }
        location /api {
        proxy_pass http://10.0.2.197:5000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
    }

        # pass PHP scripts to FastCGI server
```

```
    try {
      //delete req to the server with data payload
      const res = await axios.request({
        method: "delete",
        url: "/api",
        data: { id }
      });
```

Screenshot Placeholder

```
^C
ubuntu@ip-10-0-2-197:~/3tier-monitoring/server$ pm2 list
```

id	name	namespace	version	mode	pid	uptime	↺	status	cpu	mem	user	watching
1	backend-api	default	1.0.0	fork	8903	23m	0	online	0%	68.2mb	ubuntu	disabled
0	index	default	1.0.0	fork	8614	79m	0	online	0%	69.9mb	ubuntu	disabled

```
ubuntu@ip-10-0-2-197:~/3tier-monitoring/server$
```

---

6. Database Configuration

MySQL was installed on the database server.

Steps performed:

```
sudo apt install mysql-server
```

Database created:

```
CREATE DATABASE appdb;
CREATE USER 'appuser'@'%' IDENTIFIED BY 'password';
GRANT ALL PRIVILEGES ON appdb.* TO 'appuser'@'%';
FLUSH PRIVILEGES;
```

Screenshot Placeholder

```
ubuntu@ip-10-0-2-60:~$ sudo mysql
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.45-0ubuntu0.24.04.1 (Ubuntu)

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE appdb;
Query OK, 1 row affected (0.01 sec)

mysql> CREATE USER 'appuser'@'%' IDENTIFIED BY 'password';
Query OK, 0 rows affected (0.02 sec)

mysql> GRANT ALL PRIVILEGES ON appdb.* TO 'appuser'@'%';
Query OK, 0 rows affected (0.01 sec)

mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.00 sec)

mysql>
```

```
# localhost which is more compatible and is not less secure.
bind-address            = 0.0.0.0
mysqlx-bind-address     = 127.0.0.1
#
# * Fine Tuning
#
key_buffer_size         = 16M
```

---

7. Monitoring Implementation

7.1 Prometheus Installation

Prometheus was installed to collect metrics from all servers.

Metrics collected include:

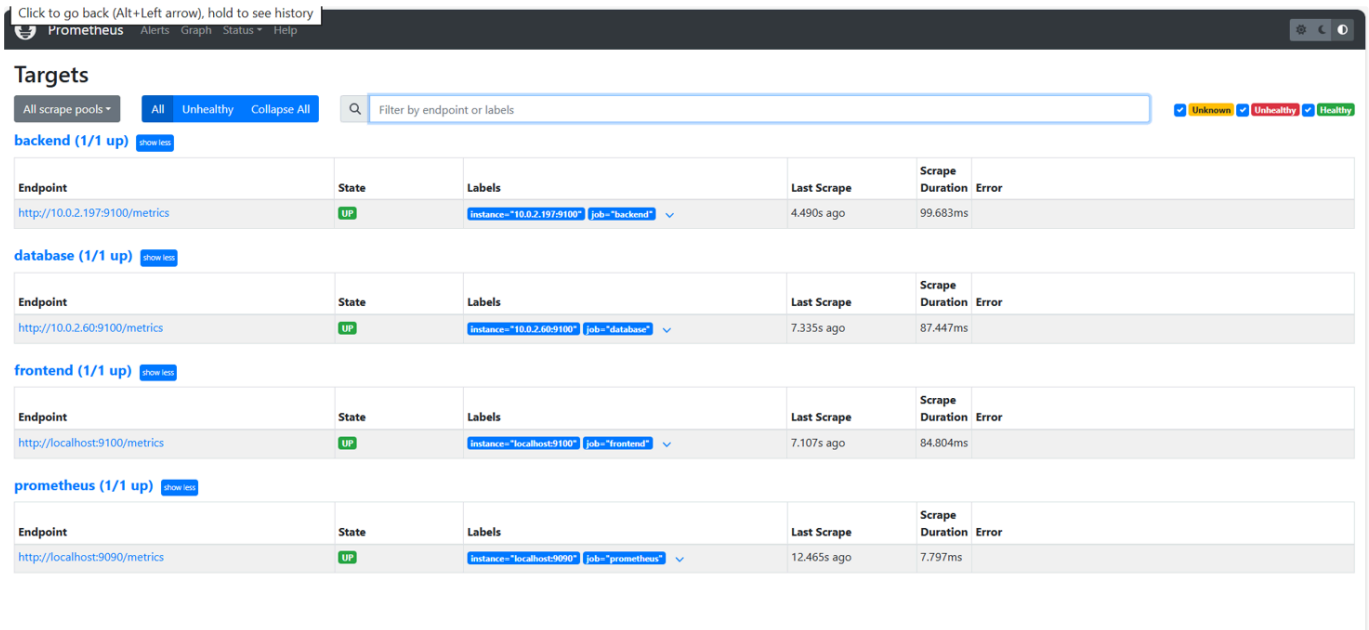
- CPU usage
- Memory usage
- Disk usage
- Network traffic

Prometheus configuration included scraping targets from:

- Frontend server
- Backend server

- Database server

Screenshot Placeholder



---

8. Visualization using Grafana

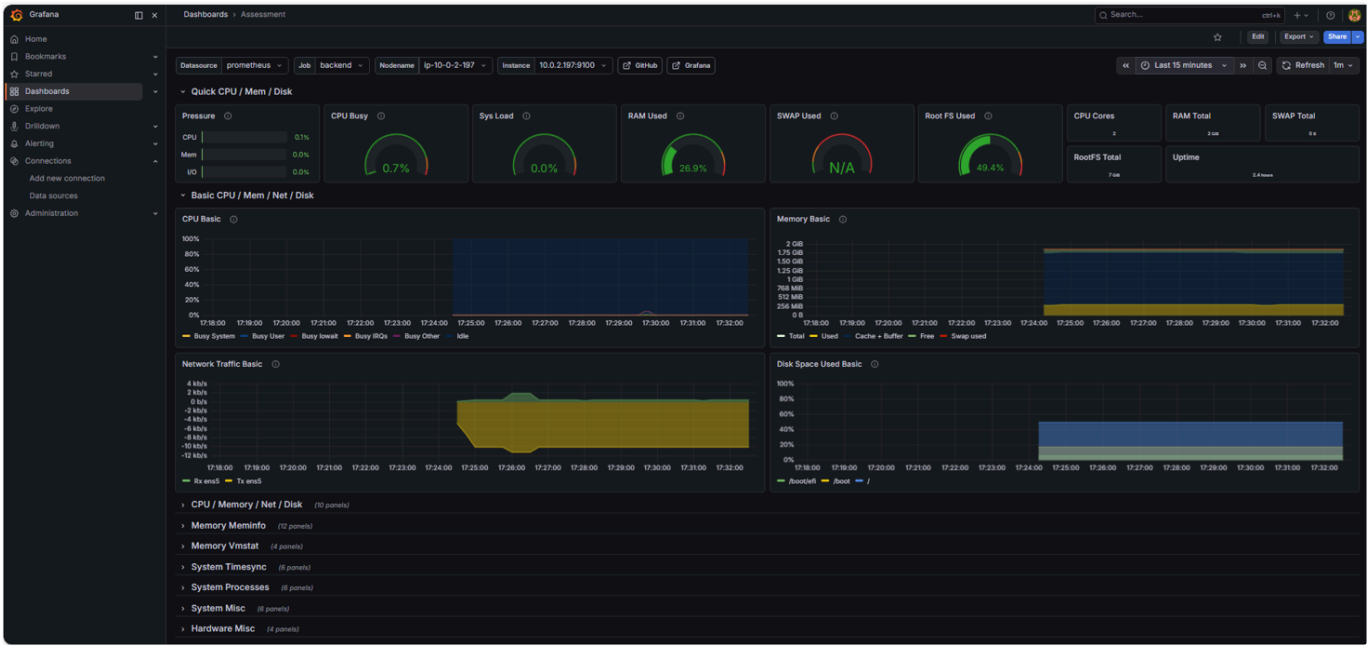
Grafana was installed to visualize Prometheus metrics.

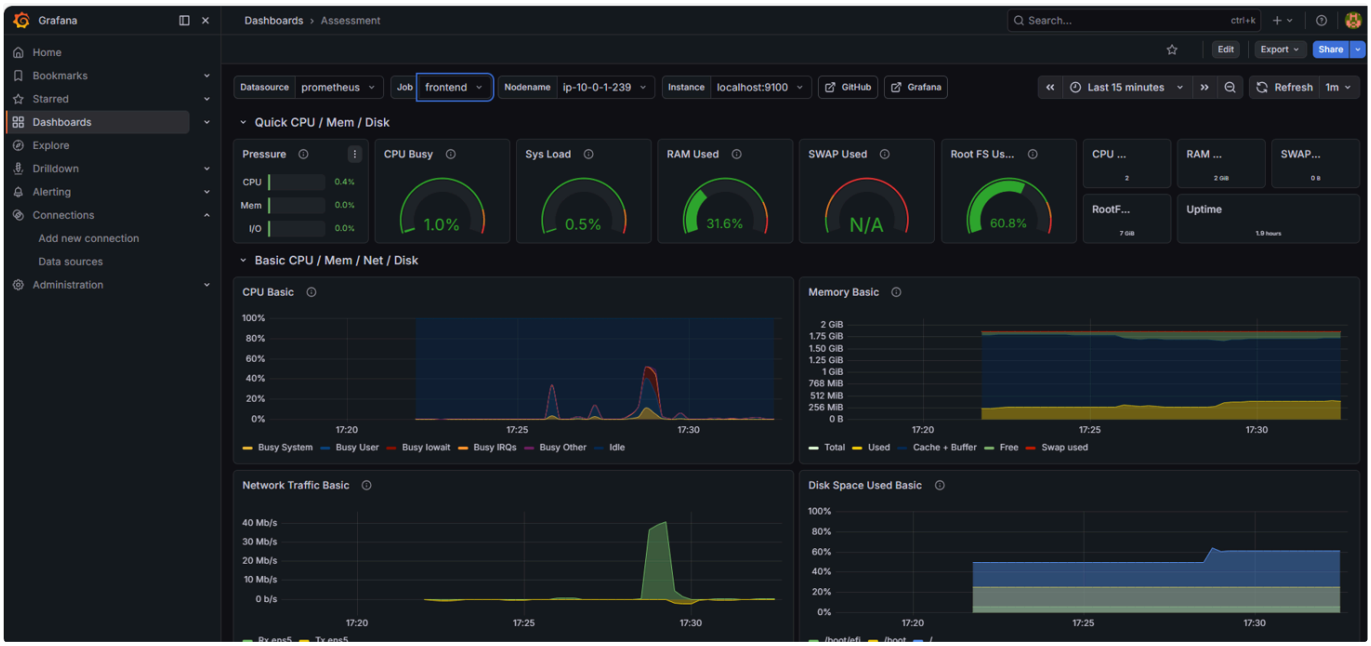
The **Node Exporter Dashboard (ID 1860)** was imported.

Metrics displayed:

- CPU utilization
- Memory usage
- Disk I/O
- Network bandwidth

Screenshot Placeholder





9. Log Aggregation using Loki

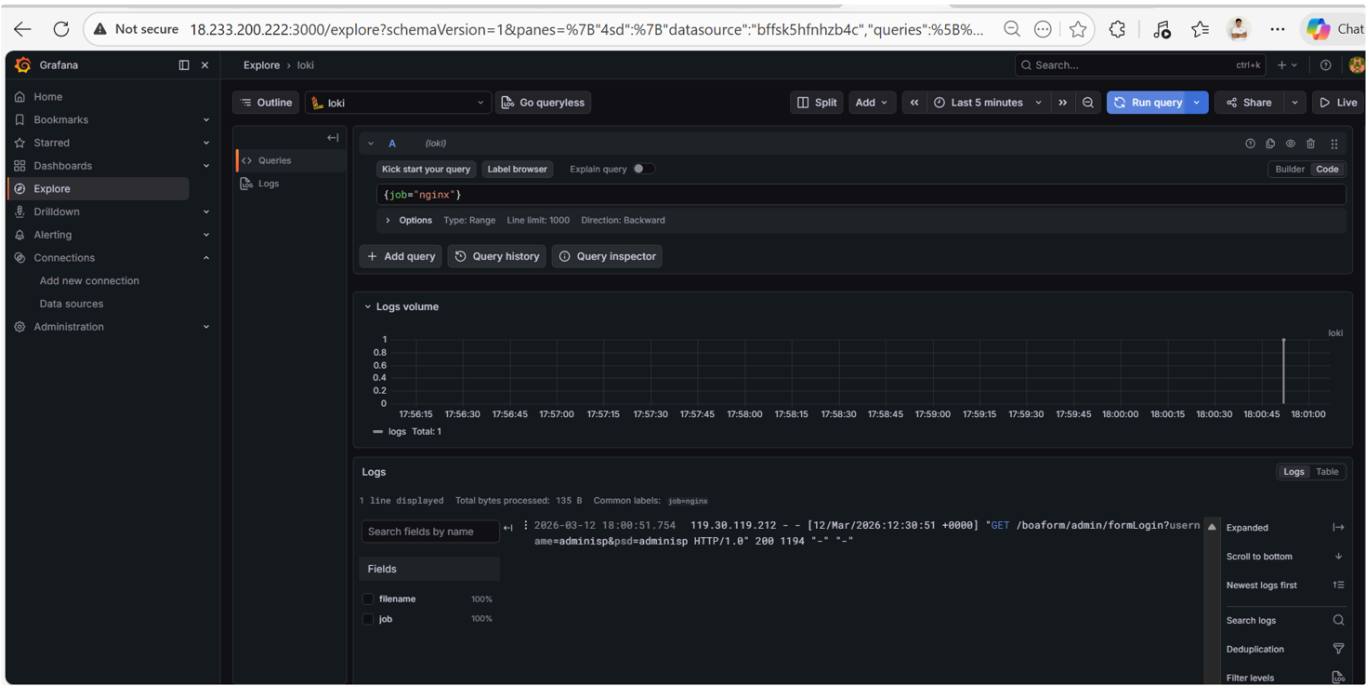
Loki was deployed to collect application logs.

Promtail was installed on the frontend server to collect logs from:

```
/var/log/nginx/access.log
/var/log/nginx/error.log
```

These logs were sent to Loki and visualized in Grafana.

Screenshot Placeholder



10. Alerting System

Alerting was implemented using:

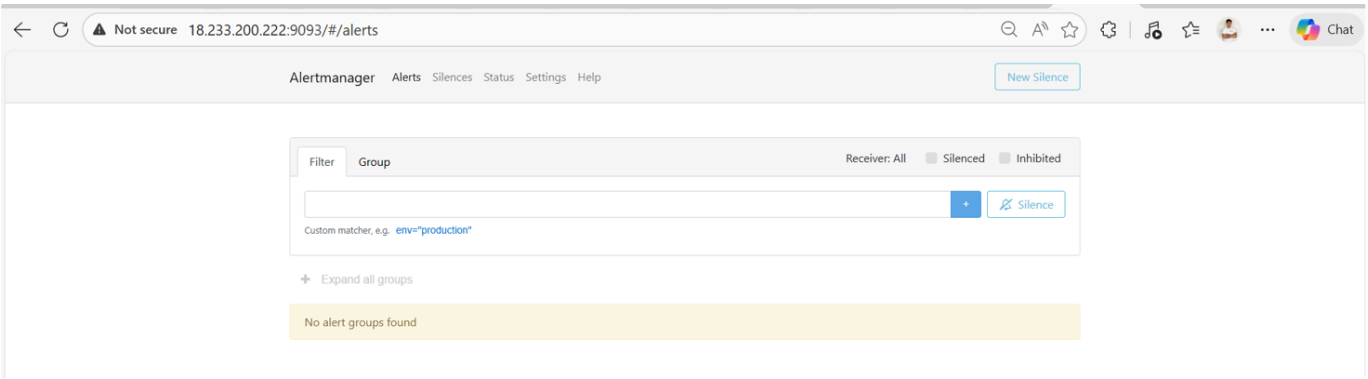
- Prometheus
- Alertmanager
- AWS SNS

Alerts configured include:

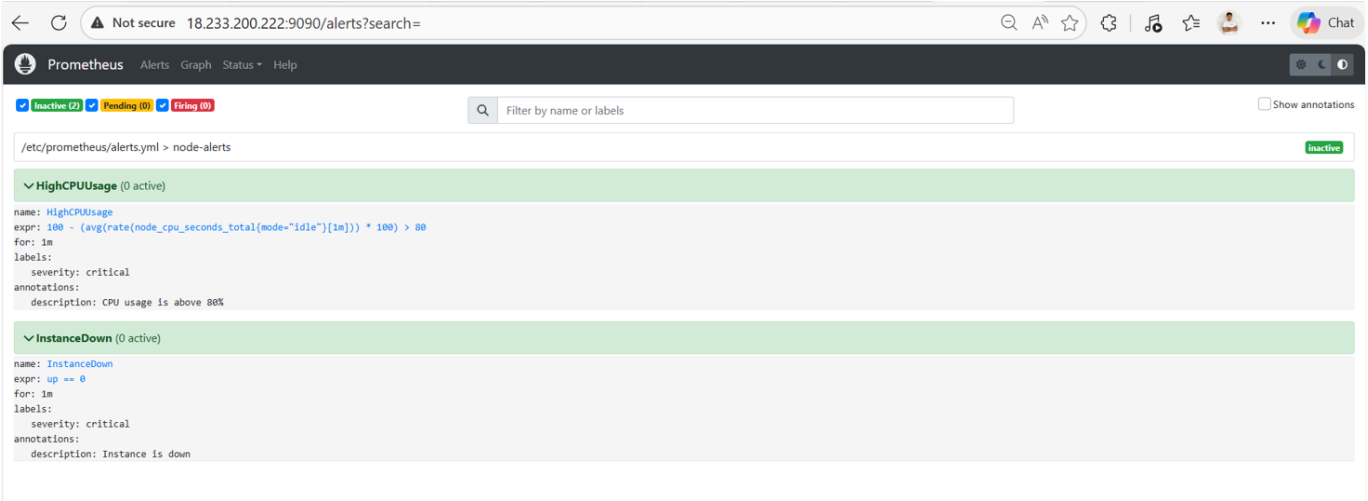
Alert	Condition
High CPU Usage	CPU > 80%
Instance Down	Server unreachable

Alerts trigger email notifications through AWS SNS.

Screenshot Placeholder




```
  GNU nano 7.2                                        /etc/prometheus/alerts.yml
groups:
- name: node-alerts
  rules:

  - alert: HighCPUUsage
    expr: 100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[1m])) * 100) > 80
    for: 1m
    labels:
      severity: critical
    annotations:
      description: "CPU usage is above 80%"

  - alert: InstanceDown
    expr: up == 0
    for: 1m
    labels:
      severity: critical
    annotations:
      description: "Instance is down"
```



---

11. HTTPS Security Implementation

HTTPS was enabled to secure communication between users and the application.

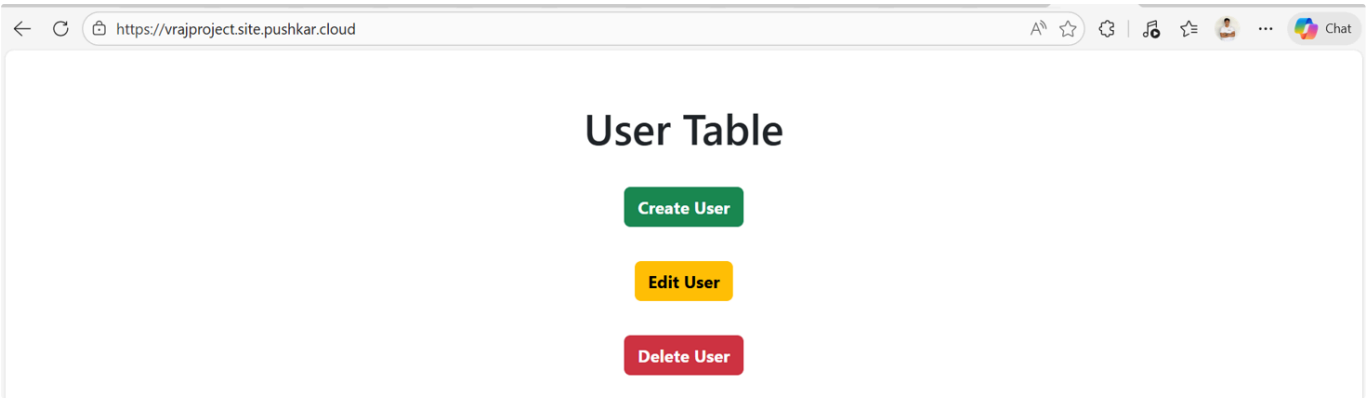
A TLS certificate was configured using Certbot.

HTTPS ensures:

- Secure encrypted communication
- Data protection
- Improved application security

Screenshot Placeholder

```
ubuntu@ip-10-0-1-239:~$ sudo certbot --nginx
Saving debug log to /var/log/letsencrypt/letsencrypt.log
Please enter the domain name(s) you would like on your certificate (comma and/or
space separated) (Enter 'c' to cancel): vrajproject.site.pushkar.cloud
Requesting a certificate for vrajproject.site.pushkar.cloud

Successfully received certificate.
Certificate is saved at: /etc/letsencrypt/live/vrajproject.site.pushkar.cloud/fullchain.pem
Key is saved at:         /etc/letsencrypt/live/vrajproject.site.pushkar.cloud/privkey.pem
This certificate expires on 2026-06-10.
These files will be updated when the certificate renews.
Certbot has set up a scheduled task to automatically renew this certificate in the background.

Deploying certificate
Successfully deployed certificate for vrajproject.site.pushkar.cloud to /etc/nginx/sites-enabled/default
Congratulations! You have successfully enabled HTTPS on https://vrajproject.site.pushkar.cloud

- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
If you like Certbot, please consider supporting our work by:
 * Donating to ISRG / Let's Encrypt:   https://letsencrypt.org/donate
 * Donating to EFF:                    https://eff.org/donate-le
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
ubuntu@ip-10-0-1-239:~$
```



---

12. Application Uptime Monitoring

To ensure the availability and reliability of the deployed application, **UptimeRobot** was used to monitor the application URL continuously.

UptimeRobot is a monitoring service that periodically checks whether a website or service is accessible. If the application becomes unavailable, it sends alerts via email.

12.1 Monitoring Configuration

The application URL was configured in UptimeRobot to monitor service availability.

Monitoring details:

Parameter	Value
Monitor Type	HTTP(s)
Monitoring Interval	5 minutes
Target URL	Application Public URL
Alert Type	Email Notification

The monitoring service checks the application every **5 minutes** to ensure it is running and accessible.

12.2 Monitoring Process

The monitoring process works as follows:

```
UptimeRobot
     ↓
Checks application URL
     ↓
If application responds → Status UP
If application fails → Alert Email Sent
```

This allows quick detection of downtime or service interruptions.

12.3 Benefits of Uptime Monitoring

Implementing uptime monitoring provides several benefits:

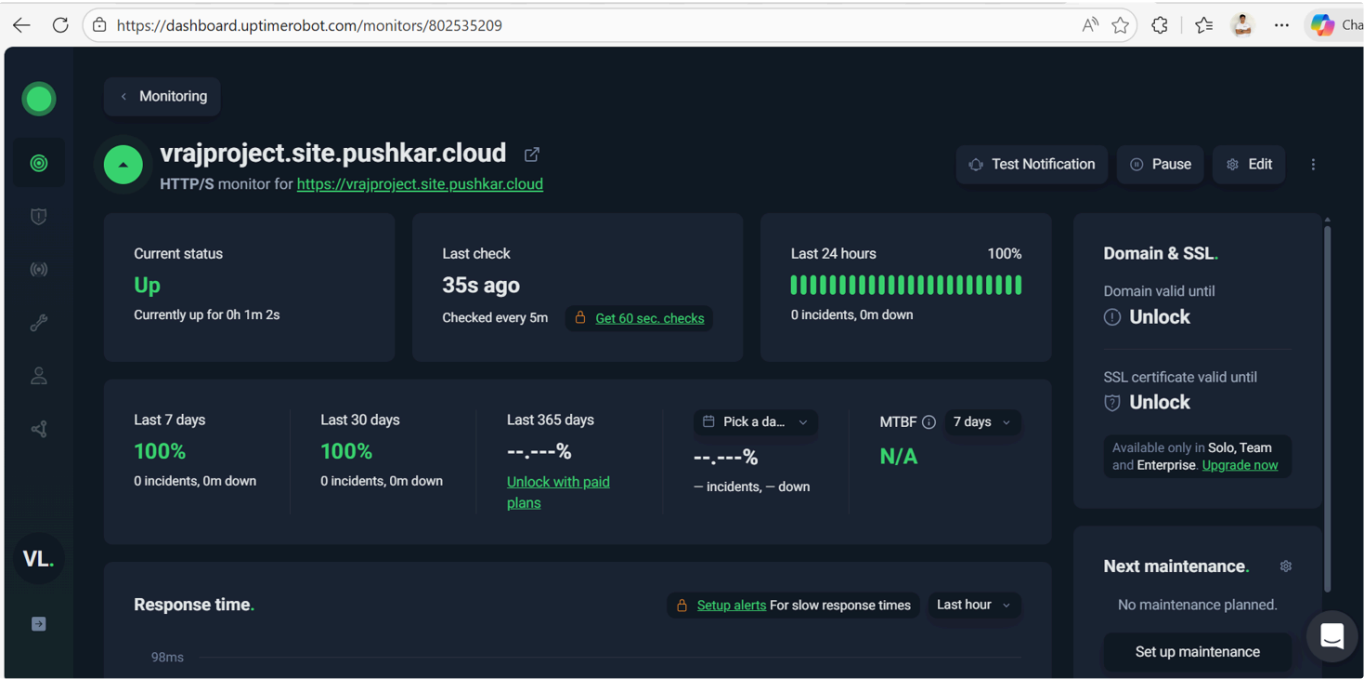
- Continuous monitoring of application availability
- Immediate alert when the service goes down
- Improved reliability and system observability
- Quick response to potential failures

12.4 Monitoring Dashboard

The UptimeRobot dashboard provides:

- Real-time status of the application
- Response time monitoring
- Historical uptime statistics
- Incident alerts

Screenshot Placeholder



---

13. Monitoring Stack Overview

The monitoring system consists of:

Tool	Purpose
Prometheus	Metrics collection
Grafana	Data visualization
Loki	Log storage
Promtail	Log collection
Alertmanager	Alert routing
SNS	Email notifications
UptimeRobot	Application uptime monitoring

---

14. Final System Architecture

```
User
 ↓
Internet
 ↓
Frontend EC2 (React + Nginx)
 ├── Prometheus
 ├── Grafana
 ├── Loki
```

```
 ├── Promtail
 ↓
Backend EC2 (Node.js API)
 ↓
Database EC2 (MySQL)

Monitoring
 ├── Prometheus
 ├── Grafana
 ├── Loki
 └── UptimeRobot
```

---

15. Conclusion

The three-tier application was successfully deployed on AWS following DevOps best practices.

Key achievements of the project include:

- Secure infrastructure deployment
- Fully functional web application
- Real-time monitoring and visualization
- Centralized log management
- Automated alert notifications

The system demonstrates the use of modern DevOps tools for building reliable and observable cloud infrastructure.

---

16. Future Improvements

Potential improvements include:

- CI/CD pipeline implementation
- Auto-scaling infrastructure
- Load balancing
- Containerization using Docker
- Kubernetes orchestration